Document d'Architecture Technique

Billetterie des Écrans de Villeneuve

Équipe : Équipe 2 — Les Écrans de Villeneuve

Membres : Sofia Selloum · Rodolphe Fanneau de la Horie · Credo Olouwachegoun

Version : 1.0 — tag v1.0 du dépôt

Date : novembre 2026 — restitution du J5

Ce DAT en une page

- **Le problème.** En mars, 14 places d'une avant-première ont été vendues deux fois. Cinq canaux et deux partenaires vendent les mêmes fauteuils ; le logiciel de caisse n'est plus maintenu après 2027.
- **Le core domain.** La Vente de places : plan de salle, blocage pendant le paiement, commande, paiement PayVia, ventes partenaires. Une place n'est vendue qu'une fois (B1).
- **L'architecture.** Un seul déploiement Spring Boot, un package par contexte. La Vente en hexagonal + DDD tactique ; les autres contextes volontairement simples, d'après la grille de décision.
- **Les trois décisions.** D1 : la Tarification est un contexte à part · D2 : les Abonnements restent en architecture simple, paramétrée · D3 : verrou optimiste par séance, avec ré-essais et critère de bascule (ADR-0001, ADR-0002).
- **Les preuves.** 90 tests verts dont 11 tests HTTP, les recherches de l'étape 6 automatisées, des diagrammes aux noms du code, une API démontrable dans Swagger et une page de réservation.
- **Le dépôt.** github.com/rdolfe/ecrans-villeneuve-equipe2 (tag v1.0) — lancer : `mvn test`, puis `mvn spring-boot:run` et http://127.0.0.1:8080/swagger-ui.html

1. Introduction et objectifs

Le produit

La billetterie des Écrans de Villeneuve vend les 1 450 fauteuils numérotés d'un multiplexe indépendant de 8 salles (35 séances par jour, 520 000 entrées par an) sur cinq canaux : le site, l'application, 4 bornes, 3 guichets et deux plateformes partenaires, SortiesFlash et CinéMalin.

Elle remplace un logiciel de caisse acheté en 2011, dont le support s'arrête fin 2027, et un site en marque blanche facturé 0,35 € par billet. La direction veut sa propre billetterie, pour au moins dix ans.

Son cœur est la **vente de places** : bloquer les fauteuils choisis le temps du paiement, encaisser par PayVia, accepter les ventes déjà payées des partenaires, et garantir qu'une place n'est jamais vendue deux fois, ce qui s'est produit en mars. Autour : la tarification, les abonnements, la programmation, le contrôle d'accès hors ligne et la déclaration hebdomadaire au CNC.

Les utilisateurs

Utilisateur	Ce qu'il fait avec le système	Canal
Client	Choisit ses places, paie, annule (B9), reçoit ses billets QR par email et SMS	Site, appli, 4 bornes (C5)
Caissier	Vend pour un client présent, encaisse par carte ou en espèces, imprime les billets	3 guichets
Abonné	Souscrit, réserve sa place gratuite (B7), annule sans pénalité (B16)	Site, appli, guichet
Agent d'accueil	Scanne les billets à l'entrée, même sans réseau (B12)	Douchettes
Programmateur (Julien)	Publie et annule les séances	Back-office
Marketing (Sabine)	Saisit la grille, les opérations et les formules d'abonnement	Back-office
Responsable financier (Marc)	Relit et dépose la déclaration CNC, suit les reversements des partenaires	Back-office, portail CNC
Partenaires	Envoient des ventes déjà payées, dans leur format (B11)	API SortiesFlash, CinéMalin

Les objectifs de qualité

Priorité	Objectif	Mesure	D'où vient-il ?
1	**Intégrité de la vente** : un fauteuil n'est jamais vendu deux fois, même au pic	0 place vendue deux fois, tous canaux confondus, au test de charge « avant-première » (420 places en 5 min, partenaires compris) ; moins de 1 % de réservations refusées pour un conflit technique	Incident de mars (Hélène), B1, pic de charge (3.7)
2	**Évolutivité par une petite équipe** : changer vite de règles et de partenaires	Un nouveau partenaire branché en moins de 2 semaines sans modifier `domaine/` ni `application/` ; une formule lancée en moins d'un mois par paramétrage ; les règles du cœur testées sans Spring ni base en moins de 10 s	Hélène (partenaire, formule, 2 développeurs), Nadia (règles changées plusieurs fois par an)
3	**Disponibilité aux heures de pointe**	Vente en ligne disponible à 99,9 % le mercredi après-midi et le week-end, aucun déploiement sur ces créneaux ; contrôle d'accès opérationnel 4 h sans réseau	Précisions 3.4, Karim (avant-première), B12

2. Contraintes et hypothèses

Contraintes

Code	Contrainte	Conséquence sur l'architecture
C1	Java 21, Spring Boot, PostgreSQL : ce que connaissent les deux développeurs	Un seul déploiement Spring Boot (ADR-0001). Spring reste dans les adapters et la configuration : domaine et application en Java pur, vérifié à chaque build (ArchitectureTest). PostgreSQL porte le verrou optimiste de la séance (ADR-0002).
C2	Hébergement chez un hébergeur cloud situé en France ; données des abonnés soumises au RGPD	Hébergeur français. Données personnelles confinées au contexte Abonnements (schéma dédié) ; les événements de vente ne transportent que des identifiants ; anonymisation automatique 3 ans après la fin de l'abonnement (3.4).
C3	Paiement par PayVia, contrat de 4 ans	PayVia derrière le port `Paiements` et l'ACL `WebhookPayVia` : son vocabulaire n'entre pas dans le domaine. Notifications « au moins une fois » → idempotence (B14) ; lien valable 15 min pour un blocage de 10 → paiement tardif (B13).
C4	Déclaration hebdomadaire des entrées au CNC, format imposé, avant le jeudi midi	Contexte Déclaration CNC en Conformist. Il lit les événements de la Vente : billets non remboursés, places d'abonnés, prix payé chez le partenaire (`VentePartenaireEnregistree`). Fichier produit le mercredi, relu et déposé par Marc.
C5	Les 4 bornes du hall sont conservées : elles affichent une page web plein écran	Les bornes sont un canal web comme le site : même API REST (`ReservationController`), aucun adapter spécifique.
C6	Mise en service avant la fin du support de l'ancien logiciel (fin 2027)	Priorité au core (Vente) ; contextes supporting simples ; génériques achetés (PayVia, service d'envoi). Bascule progressive par canal : site, puis bornes, puis guichets.

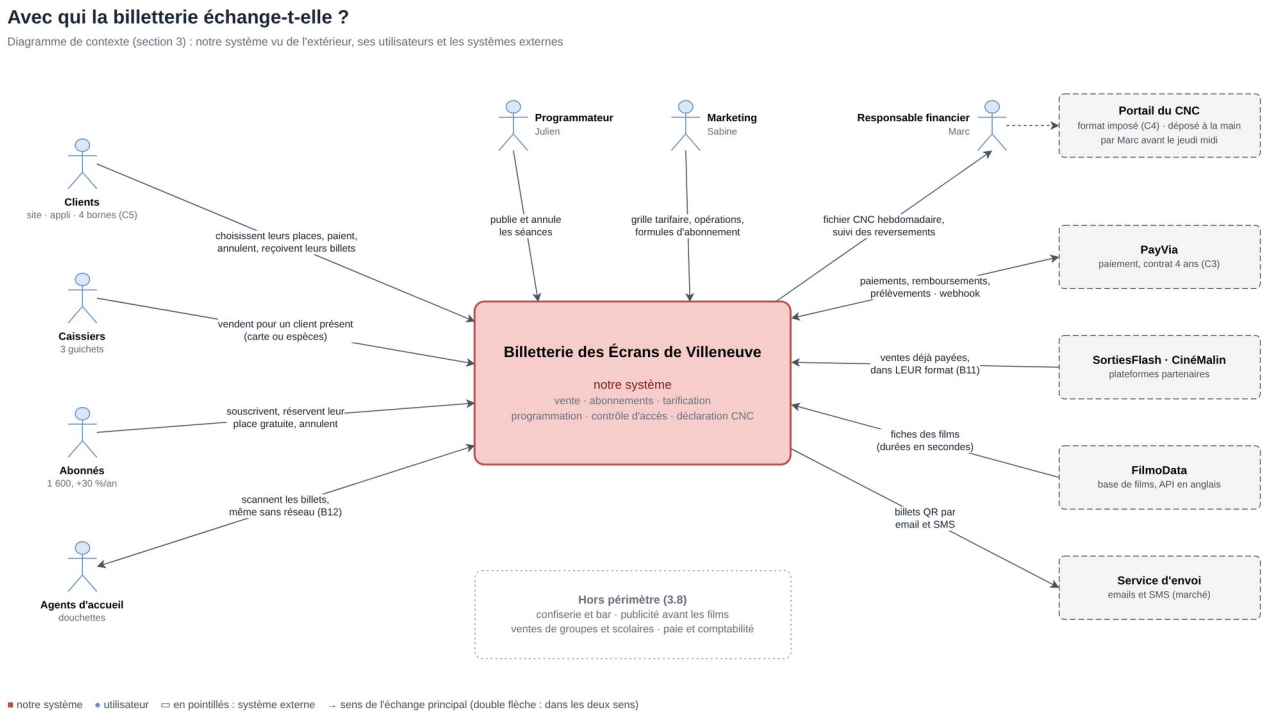
Hypothèses sur le brief

Question	Hypothèse retenue	Justification
H1 · Séance annulée (3.9-1) : qui rembourse un billet acheté chez un partenaire ?	Le partenaire rembourse son client ; nous ne lui facturons pas ce billet. La Vente publie le même `PaiementARembourser` ; le contexte Reversements le déduit au lieu d'appeler PayVia.	Le partenaire a encaissé lui-même (B11, Marc) : nous n'avons ni son moyen de paiement ni son client. L'argent ne nous est pas encore parvenu (reversement mensuel) : il suffit de ne pas le réclamer.
H2 · Un abonné résilie (3.9-2) : ses places gratuites du mois suivant restent-elles valables ?	La résiliation prend effet à la fin du mois (3.4). Les places des séances jusqu'à cette date restent valables ; celles d'après sont annulées automatiquement, sans no-show, et l'abonné est prévenu.	Le droit à la place gratuite vient de l'abonnement actif à la date de la séance (B7). Libérer ces fauteuils les remet en vente.
H3 · Guichet, paiement en espèces (3.9-3) : places bloquées 10 minutes ?	Oui : même cas d'usage `ReserverDesPlaces` que les autres canaux, puis le caissier confirme l'encaissement aussitôt (adapter « caisse » qui appelle `ConfirmerPaiement`). Si le client part sans payer, le blocage expire seul (B2).	B1 vaut « tous canaux confondus » : un seul chemin de vente, pas d'exception guichet. Pendant que le client compte sa monnaie, sa place ne doit pas partir sur le site.
H4 · Douchettes hors ligne (3.9-4) : combien de temps, avec quelles données ?	4 heures d'autonomie. Chaque douchette télécharge à l'ouverture, puis chaque minute, les billets valides du jour : identifiant, séance, place, fenêtre de validité (B12), abonné oui/non, sans donnée personnelle.	Karim accepte un double scan rarissime, pas une entrée bloquée. 4 h couvrent la plus longue soirée. Sans donnée personnelle, une douchette perdue ne pose pas de problème RGPD (C2). Les no-shows (B8) attendent la synchronisation.

Question	Hypothèse retenue	Justification
	Les QR sont signés : un billet vendu pendant la panne se vérifie quand même. Les scans hors ligne sont synchronisés au retour du réseau.	
H5 · Comment se cumulent suppléments et opérations ?	Prix = catégorie + suppléments (3D +2 €, Grand Large +3 €, cumulables). Une opération fixe un prix final, suppléments compris. La place d'abonné reste à 0 €.	Lecture de 3.4 et de Sabine : « le prix affiché au client doit être celui qu'il paie ».
H6 · Une séance déplacée (B10)	Traitée comme une annulation suivie d'une nouvelle séance (nouvel identifiant), remise en vente.	Un client qui avait choisi 20 h 30 n'a pas forcément choisi 22 h : il est remboursé et peut racheter.
H7 · Une vente partenaire arrive après la fermeture (B4)	Elle est refusée (422), comme toute vente.	B11 : « elle respecte B1 à B4 ».
H8 · Heure de référence	Heure de Paris, fournie par une `Clock` injectée ; jamais `LocalDateTime.now()` sans horloge.	Testabilité (HorlogeReglable) et cohérence entre canaux ; vérifié par ArchitectureTest.

3. Contexte

Avec qui la billetterie échange-t-elle ? Le système est un seul rectangle ; à gauche et en haut ses utilisateurs, à droite les systèmes externes.



Acteur ou système externe	Ce qu'il échange avec le système
Clients (site, appli, bornes)	Choix des places, réservation, paiement par PayVia, annulation (B9) ; reçoivent leurs billets QR
Caissiers	Ventes au guichet (carte, espèces), impression des billets
Abonnés	Souscription, réservation de la place gratuite (B7), annulation (B16)
Agents d'accueil (douchettes)	Téléchargent la liste des billets valides, renvoient les scans (B12)
Programmateur · Marketing · Responsable financier	Séances ; grille, opérations et formules ; fichier CNC hebdomadaire et reversements
PayVia	Demandes de paiement, notifications (webhook), remboursements, prélèvements des abonnés (annexe A)
SortiesFlash · CinéMalin	Ventes déjà payées dans leur format (annexe B), confirmations ; reversement mensuel (grille – 8 %)
FilmoData	Fiches des films (API en anglais, durées en secondes)
Service d'envoi	Emails et SMS : billets, annulations, remboursements
Portail du CNC	Déclaration hebdomadaire, déposée à la main par Marc avant le jeudi midi

Hors périmètre (3.8) : confiserie et bar, publicité avant les films, ventes de groupes et scolaires, paie et comptabilité.

4. Domaine

Glossaire

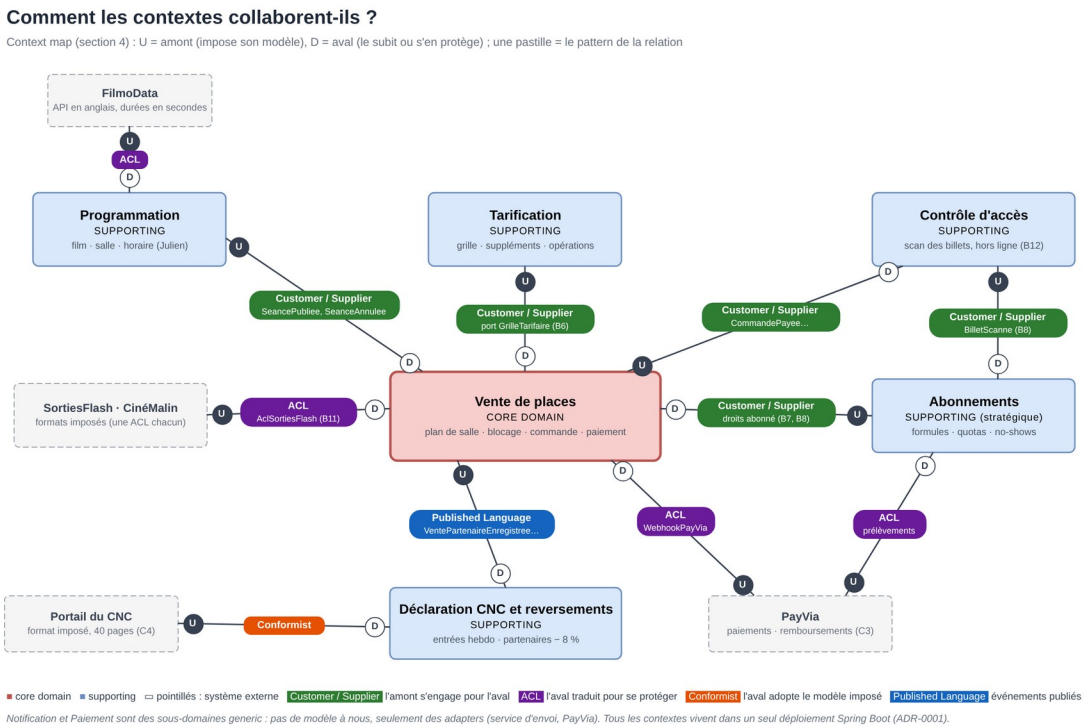
Terme	Contexte	Définition
Séance ⚠️	Programmation	Un film projeté dans une salle à un horaire (Julien). Publiée le mardi soir, annulable.
Séance ⚠️	Vente	Le plan de salle d'une séance à vendre : ses fauteuils, libres, bloqués ou vendus (agrégat `Seance`). Ni film, ni durée.
Place	Vente	Un fauteuil numéroté du plan de salle : un rang et un numéro (`PlaceId` « G7 »).
Séance pleine	Vente	Plus aucun fauteuil libre sur le plan de salle (Nadia). Pas une jauge.
Blocage	Vente	Les places choisies, retenues pour une commande pendant 10 minutes, le temps du paiement ; libérées seules (B2).
Réservation ⚠️	Vente	Le choix des places : blocage + commande en attente de paiement (`ReserverDesPlaces`). Pas encore un achat.
Réservation ⚠️	Abonnements	Une place gratuite d'abonné pour une séance à venir, comptée dans son quota (B7 : une par séance, deux séances à venir).
Commande	Vente	1 à 10 billets d'une même séance, payés en une fois (B3) : en attente, payée, expirée ou à rembourser.
Billet ⚠️	Vente	Le droit d'entrer pour une place, à un tarif figé ; il vit dans la commande.
Billet ⚠️	Contrôle d'accès	Un QR code valable de 30 min avant à 15 min après le début, scanné une seule fois (B12).
Tarif	Vente	Une catégorie et le prix figé au choix des places, garanti jusqu'au paiement (B6).
Grille tarifaire	Tarification	Les prix par catégorie (plein, réduit, enfant, matin, abonné) et les suppléments 3D et Grand Large.
Opération	Tarification	Un prix événementiel qui remplace la grille pendant une période (Fête du cinéma à 5 €).
Formule	Abonnements	Une offre d'abonnement (Villeneuve Illimité…), paramétrée par le marketing.
No-show	Abonnements	Billet d'abonné non scanné à la fin de sa fenêtre de validité, sauf séance annulée (B8) ou place annulée à temps (B16).
Suspension	Abonnements	7 jours sans réservation à l'avance après 3 no-shows en 30 jours (B8).
Annulation ⚠️	Vente · Programmation · Abonnements	Trois sens : le client annule sa commande (B9, jusqu'à 2 h avant) ; le cinéma annule une séance (B10) ; l'abonné annule sa place gratuite (B16, jusqu'à 30 min avant).
Vente partenaire	Vente	Vente déjà payée chez SortiesFlash ou CinéMalin, reçue en un appel, soumise à B1–B4 (B11).
Paiement tardif	Vente	Paiement reçu après la fin du blocage : on vend si les places sont libres, sinon on rembourse (B13).
Reversement	Déclaration CNC et reversements	Ce que le partenaire nous verse chaque mois : le tarif de la grille moins 8 % (B11).
Entrée	Déclaration CNC et reversements	Un billet non remboursé, places d'abonnés comprises, déclaré au prix payé par le spectateur (3.4).

⚠️ = terme qui change de sens d'un contexte à l'autre : chaque contexte a son propre modèle. Le code de la Vente utilise exactement ces mots (`Seance`, `Commande`, `Billet`, `Tarif`, `ReserverDesPlaces`…).

Sous-domaines

Sous-domaine	Type	Stratégie
Vente de places	Core	Là où se jouent le risque et la différence : B1 au pic, 5 canaux + 2 partenaires, règles changées plusieurs fois par an, expert disponible chaque semaine (Nadia). On investit : hexagonal + DDD tactique.
Abonnements	Supporting (stratégique)	« Notre arme » (Hélène), mais l'avantage tient à l'offre, pas à un algorithme : quotas et compteurs. Formules qui changent sans cesse, expert disponible une fois par mois → architecture simple et paramétrée (décision 2).
Tarification	Supporting	Calcul simple mais qui change au rythme du marketing → contexte à part, simple, paramétré (décision 1).
Programmation	Supporting	Stable depuis vingt ans, peu de règles → CRUD + ACL vers FilmoData.
Contrôle d'accès	Supporting	La difficulté est technique (hors ligne), pas métier → couches simples + application douchette.
Déclaration CNC et reversements	Supporting	Obligation réglementaire au format imposé → traitement par lots simple, Conformist.
Paiement	Generic	On achète : PayVia (C3), derrière un port et une ACL.
Notification	Generic	On achète : service d'envoi du marché, un simple adapter.

Context map



Amont (U)	Aval (D)	Pattern	Pourquoi
FilmoData	Programmation	ACL	API en anglais, durées en secondes : traduites dans notre modèle sans s'y propager.

Amont (U)	Aval (D)	Pattern	Pourquoi
Programmation	Vente	Customer / Supplier	La Vente ouvre à la publication (B4) et réagit aux annulations (B10) : la Programmation s'engage à publier SeancePubliee et SeanceAnnulee.
Tarification	Vente	Customer / Supplier	La Vente demande le prix au choix des places (port `GrilleTarifaire`) et le fige dans le billet (B6).
Abonnements	Vente	Customer / Supplier	Avant une place à 0 €, la Vente vérifie le droit de l'abonné : quota (B7), suspension (B8).
Vente	Contrôle d'accès	Customer / Supplier	Le Contrôle construit la liste des billets valides à partir de CommandePayee et PaiementARembourser.
Contrôle d'accès	Abonnements	Customer / Supplier	Le scan prouve la venue (Karim) : BilletScanne alimente le décompte des no-shows (B8).
SortiesFlash, CinéMalin	Vente	ACL	Formats imposés qu'ils ne changeront pas (annexe B) : une ACL par partenaire, un seul cas d'usage.
Vente	Déclaration CNC et reversements	Published Language	Événements publiés : la Vente ne connaît ni le CNC ni les reversements ; VentePartenaireEnregistree porte le prix payé.
Portail du CNC (format)	Déclaration CNC et reversements	Conformist	40 pages imposées, non négociables : on adopte le modèle tel quel.
PayVia	Vente	ACL	`WebhookPayVia` et le port `Paiements` traduisent event, reference, amount ; PayVia remplaçable dans 4 ans (C3).
PayVia	Abonnements	ACL	Prélèvements mensuels : même protection, côté Abonnements.

Notification et Paiement n'apparaissent pas comme contextes : ils n'ont pas de modèle à nous, seulement des adapters. La Vente rend aussi la place gratuite au quota des Abonnements par l'événement CommandeExpiree (même langage publié).

5. Choix d'architecture par contexte

Grille de décision

Contexte	Complexité	Valeur	Durée de vie	Volatilité	Experts	Canaux	Total	Architecture retenue
Vente de places	3	3	3	3	3	3	**18**	Hexagonale + DDD tactique
Abonnements	2	3	3	3	1	2	**14**	Couches + paramétrage (D2)
Tarification	2	2	3	3	1	1	**12**	Contexte à part, couches simples (D1)
Contrôle d'accès	2	2	3	1	2	2	**12**	Couches + appli douchette hors ligne
Déclaration CNC	2	2	3	1	2	1	**11**	Traitement par lots, Conformist
Programmation	1	2	3	1	1	1	**9**	CRUD + ACL FilmoData

Justification

Lecture de la grille. Chaque critère est noté de 1 (faible) à 3 (fort) ; un 3 pousse vers un modèle riche. Total ≥ 15 : hexagonal + DDD tactique ; 11 à 14 : couches simples, avec des ports vers l'extérieur ; ≤ 10 : CRUD. **Garde-fou :** un score Experts de 1 plafonne à « couches + paramétrage », car le DDD se nourrit d'échanges fréquents avec l'expert.

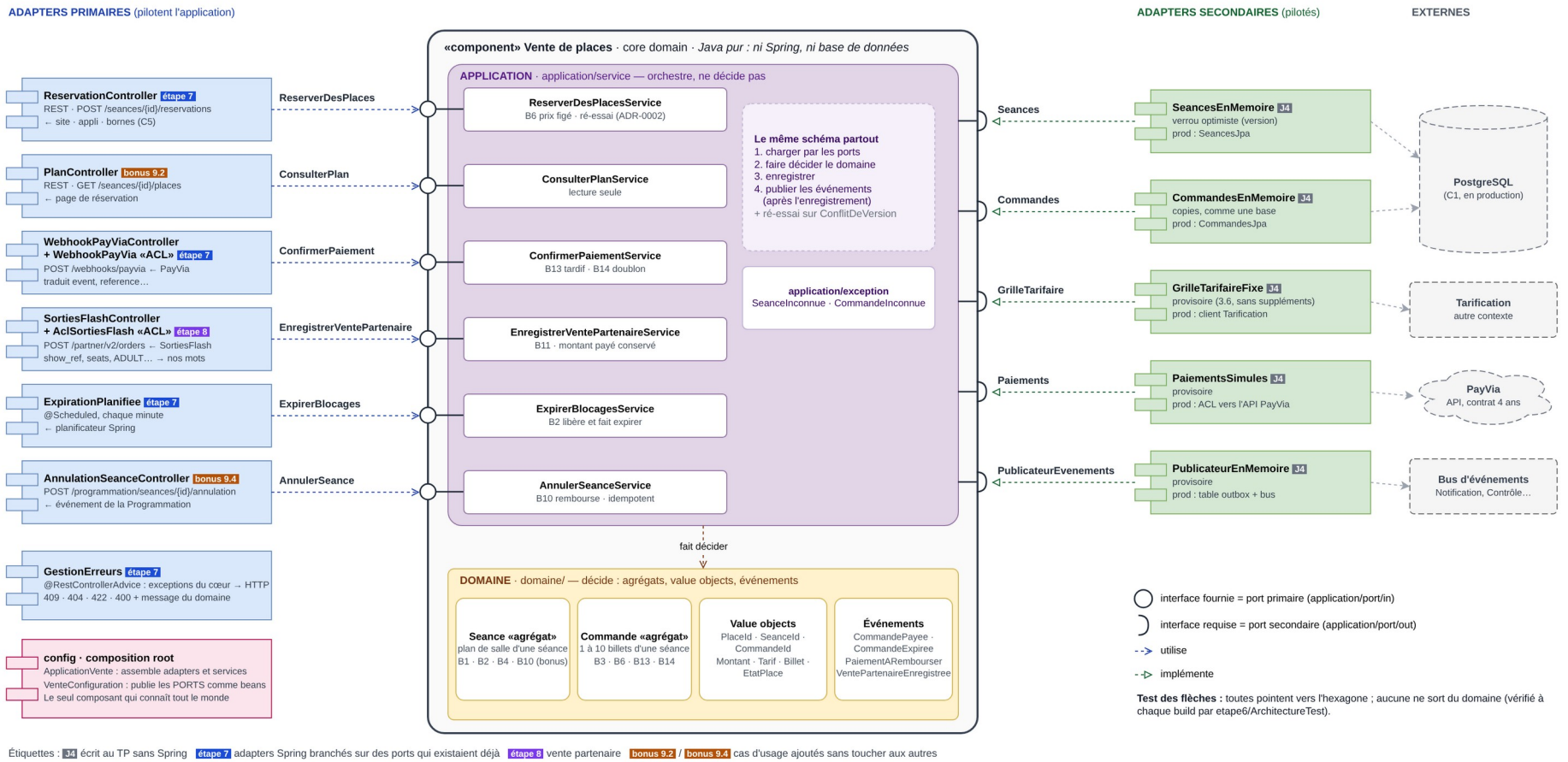
- **Vente** — complexité 3 (B1 à B6, B11, B13, B14, concurrence au pic) · valeur 3 (incident de mars, chiffre d'affaires) · durée 3 (dix ans) · volatilité 3 (« on change nos règles plusieurs fois par an ») · experts 3 (Nadia chaque mardi) · canaux 3 (site, appli, bornes, guichets, 2 partenaires, webhook, planificateur).
- **Abonnements** — complexité 2 (quotas et compteurs B7, B8, B15, B16, sans algorithme) · valeur 3 (« notre arme ») · durée 3 · volatilité 3 (formules testées en permanence) · experts 1 (Sabine une fois par mois) · canaux 2. Total 14, mais plafonné par le garde-fou : architecture simple.
- **Tarification** — complexité 2 (catégories, suppléments, opérations) · valeur 2 · durée 3 · volatilité 3 (opérations fréquentes) · experts 1 (Sabine) · canaux 1 (back-office, un seul client : la Vente).
- **Contrôle d'accès** — complexité 2 (fenêtre de validité, scan unique ; la vraie difficulté est technique : le hors-ligne) · valeur 2 · durée 3 · volatilité 1 · experts 2 (Karim sur place) · canaux 2 (douchettes, événements).
- **Déclaration CNC** — complexité 2 (format de 40 pages, mais aucune décision) · valeur 2 (obligation légale) · durée 3 · volatilité 1 (format stable) · experts 2 (Marc) · canaux 1.
- **Programmation** — complexité 1 (film, salle, horaire) · valeur 2 · durée 3 · volatilité 1 (« je programme de la même façon depuis vingt ans ») · experts 1 (Julien à distance, réponse dans la semaine) · canaux 1.

Architecture retenue. Un seul déploiement Spring Boot, un package par contexte et un schéma PostgreSQL par contexte (monolithe modulaire) ; seule la Vente est hexagonale avec un modèle riche. Les décisions 1 et 2 sont tranchées dans l'ADR-0001, la décision 3 dans l'ADR-0002.

6. Vue composants

Comment la vente de places est-elle branchée sur le monde extérieur ?

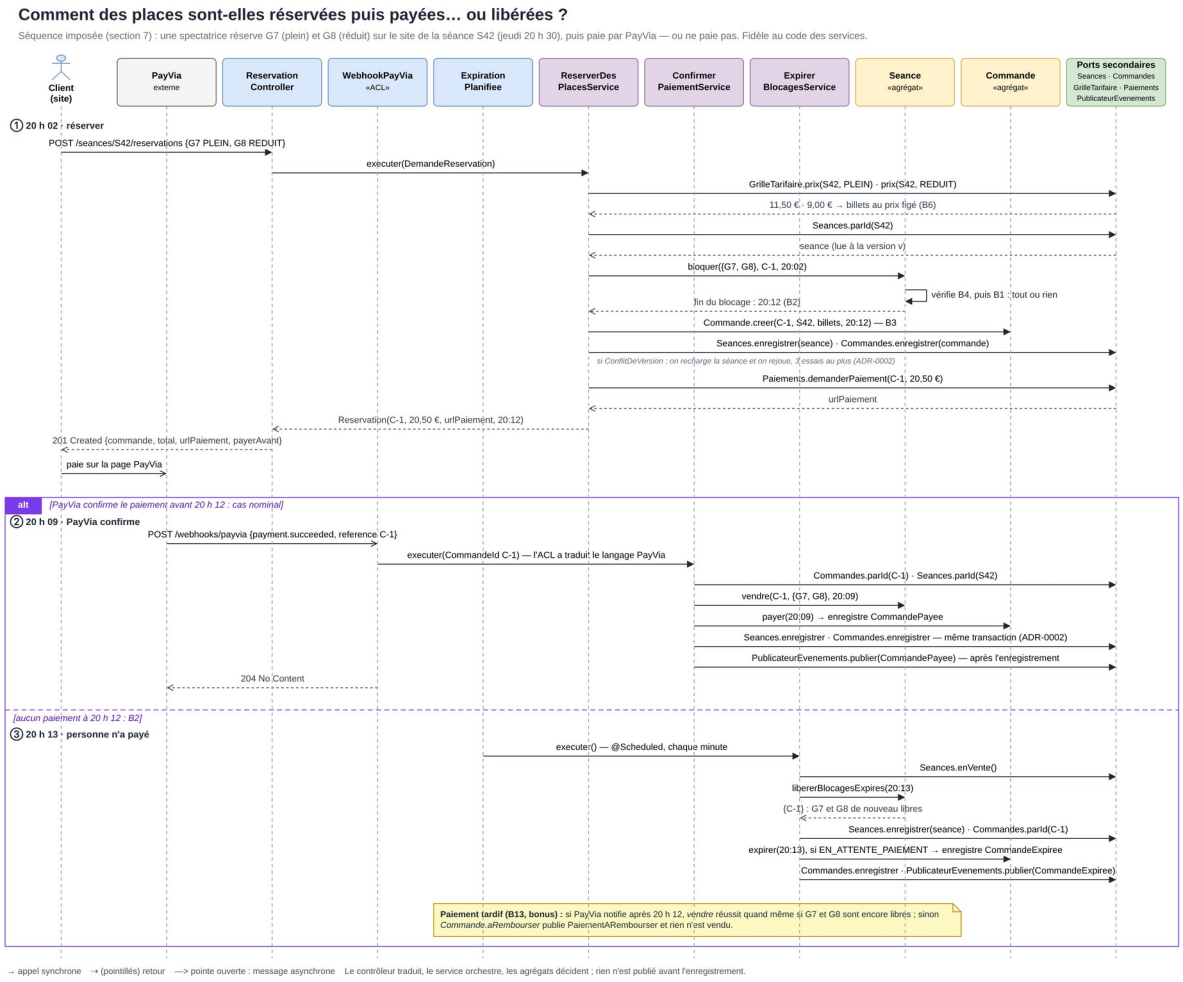
Vue composants du contexte Vente (section 6) : l'hexagone, ses ports (interfaces fournies et requises), ses adapters et les systèmes externes. Les noms sont ceux du code (com.ecrans.vente).



- **À gauche**, les adapters primaires pilotent l'hexagone par ses ports d'entrée. Les trois adapters Spring de l'étape 7 (REST, webhook, planificateur) se sont branchés sur des ports qui existaient déjà au J4 : aucune ligne du cœur n'a changé.
- **Au centre**, l'application orchestre et le domaine décide, sans Spring ni base de données. **À droite**, les adapters secondaires implémentent les ports de sortie ; ceux du TP sont en mémoire ou simulés, leur remplaçant de production est indiqué. Les ACL `WebhookPayVia` et `AclSortiesFlash` enferment le langage des partenaires.

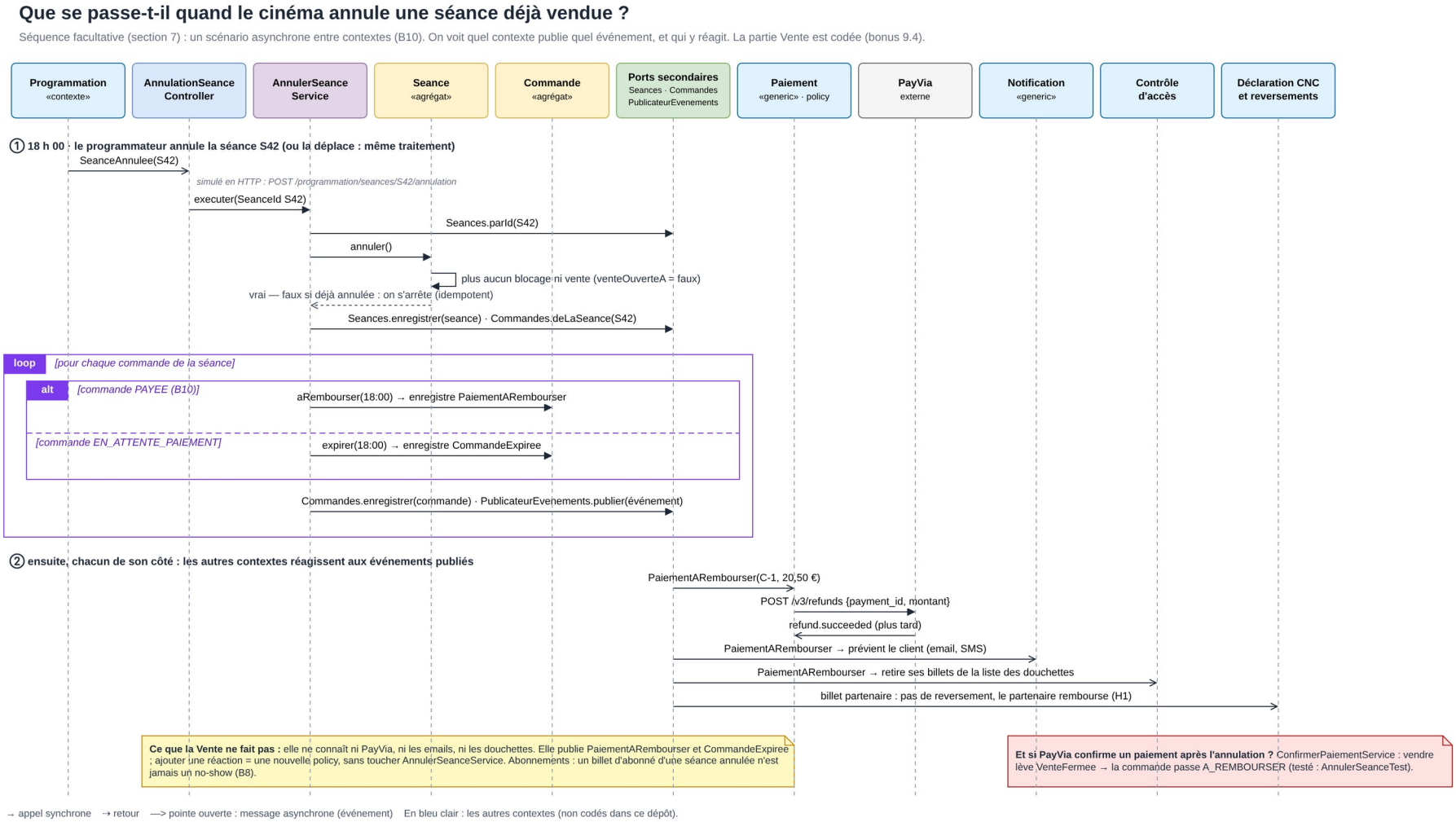
7. Vue dynamique

Réserver et payer des places



Le contrôleur traduit, le service orchestre, `Seance` et `Commande` décident. Toutes les décisions sont prises avant le premier enregistrement : une commande refusée (11 places, place prise) ne laisse rien de bloqué, et PayVia n'est appelé qu'ensuite. Le paiement vend les places et paie la commande dans la même transaction (ADR-0002). Sans paiement, le planificateur libère les places chaque minute ; un second passage ne republie rien.

Second scénario (facultatif) : le cinéma annule une séance déjà vendue

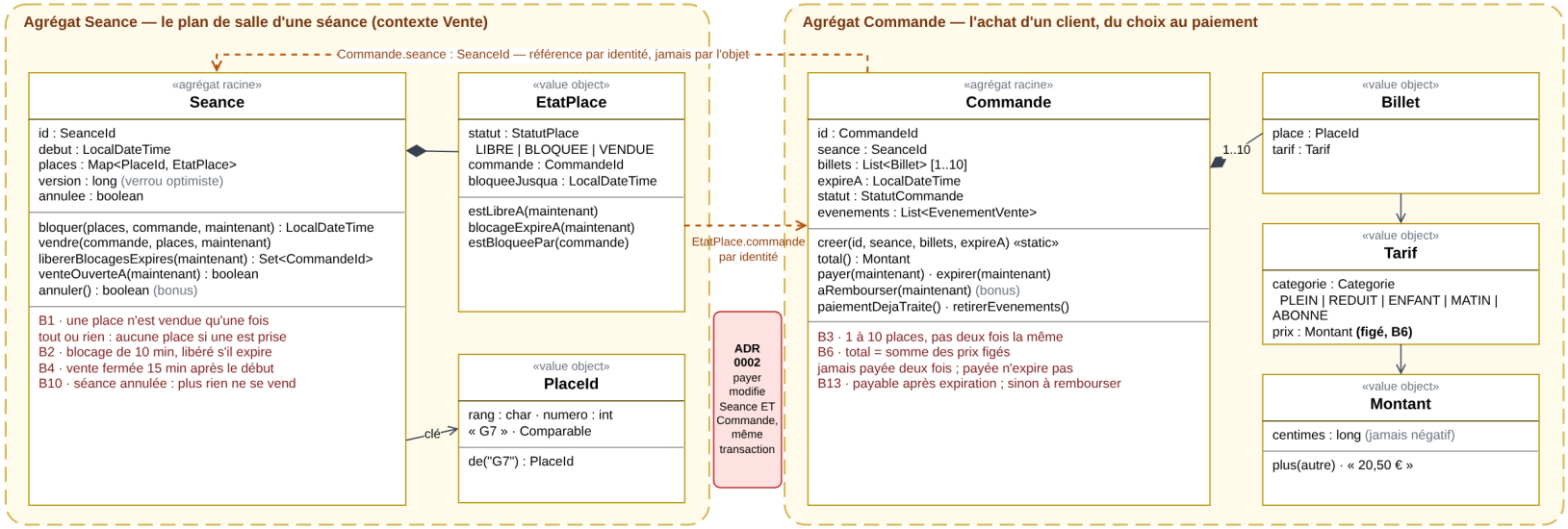


La Programmation publie, la Vente réagit seule (bonus 9.4, `AnnulerSeanceService`) et publie à son tour `PaiementARembourser` et `CommandeExpiree` ; Paiement, Notification, Contrôle d'accès et Reversements y réagissent chacun de leur côté. Une seconde annulation ne rembourse personne deux fois (AnnulerSeanceTest).

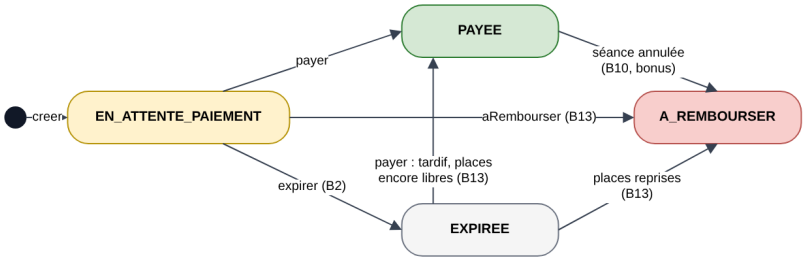
8. Modèle du domaine

Quels agrégats protègent les règles de la vente ?

Modèle du domaine (section 8) : deux agrégats qui se référencent par identité, leurs value objects, le cycle de vie d'une commande et les événements publiés



Cycle de vie d'une Commande (StatutCommande) — chaque transition est une méthode de l'agrégat



Événements de domaine (domaine/evenement) — au passé, publiés <u>après</u> l'enregistrement

- **CommandePayee** (commande, seance, places, total, survenuLe)
  → Notification envoie les billets · Contrôle d'accès les ajoute
- **CommandeExpiree** (commande, seance, survenuLe)
  → Abonnements rend la place gratuite au quota (B7)
- **PaiementARembourser** (commande, montant, survenuLe)
  → Paiement rembourse par PayVia · Notification prévient
- **VentePartenaireEnregistree** (commande, partenaire, référence, montantPaye, survenuLe)
  → Déclaration CNC (prix payé) · reversement grille − 8 % (B11)

◆ composition (vit et meurt avec la racine) → association ⇢ référence par identité entre agrégats Les invariants (en rouge) sont prouvés par les tests du tableau de la section 8.

Invariants → agrégat → test. Chaque règle est protégée par un agrégat et prouvée par un test (méthodes de test du dépôt ; « … » abrège les noms les plus longs).

Invariant (règle)	Agrégat qui le protège	Test qui le prouve
B1 · une place n'est vendue qu'une fois, tous canaux confondus	Seance (`bloquer`, `vendre`)	• SeanceTest · `refuse_une_place_vendue_B1` • ReserverDesPlacesTest · `une_place_deja_prise_…_B1` • VentePartenaireApiTest · `placeVenduePartenaireIndisponibleSurLeSite`
Tout ou rien · si une place est prise, aucune n'est bloquée	Seance (`bloquer`)	• SeanceTest · `tout_ou_rien_si_une_place_est_prise_…` • VentePartenaireApiTest · `unePlacePriseToutEstRefuse`
B2 · blocage de 10 min, libéré automatiquement	Seance (`libererBlocagesExpires`) + ExpirerBlocagesService	• SeanceTest · `bloque_des_places_libres_pendant_10_minutes_B2` • ExpirerBlocagesTest (4 tests) • ScenarioSoireeDuJeudiTest · `sans_paiement_les_places_reviennent_a_la_vente`
B3 · 10 places au plus, sans doublon	Commande (`creer`)	• CommandeTest · `refuse_plus_de_10_places_B3` • CommandeTest · `refuse_deux_fois_la_meme_place` • ReserverDesPlacesTest · `plus_de_10_places_sont_refusees_…_B3`
B4 · vente fermée 15 min après le début	Seance (`venteOuverteA`)	• SeanceTest · `la_vente_est_fermee_au_dela_B4` • SeanceTest · `la_vente_est_ouverte_jusqu_a_15_…_B4`
B6 · prix figé au choix des places	Commande (`Billet`, `Tarif`)	• CommandeTest · `le_total_est_la_somme_des_prix_figes_…_B6`
Cycle de vie · jamais payée deux fois ; une commande payée n'expire pas	Commande (`payer`, `expirer`)	• CommandeTest · `on_ne_paie_pas_deux_fois` • CommandeTest · `une_commande_payee_n_expire_pas`
B13 · paiement tardif : vendu si libre, sinon remboursé	Seance (`vendre`) + Commande (`aRembourser`)	• PaiementTardifTest · `places_encore_libres_on_vend_quand_meme` • PaiementTardifTest · `places_reprises_entre_temps_on_rembourse`
B14 · une notification reçue deux fois ne produit qu'un effet	Commande (`paiementDejaTraite`)	• PaiementEnDoubleTest · `la_deuxieme_notification_ne_change_rien`
B11 · vente partenaire : B1 à B4, montant payé conservé	Seance + Commande ; événement VentePartenaireEnregistree	• EnregistrerVentePartenaireTest (7 tests) • AclSortiesFlashTest · `conserve_le_montant_paye_…_B11`
B10 · séance annulée : plus de vente, remboursement une seule fois	Seance (`annuler`) + Commande (`aRembourser`)	• AnnulerSeanceTest (5 tests)
Concurrence · deux ventes simultanées : la seconde rejoue	Seance (`version`) + adapter Seances	• ConcurrenceTest (3 tests)

Value objects

`PlaceId` (rang + numéro, comparable : les places sont toujours traitées dans le même ordre), `SeanceId`, `CommandeId` (c'est aussi la « reference » transmise à PayVia), `Montant` (en centimes, jamais négatif : aucune

erreur d'arrondi), `Tarif` (catégorie + prix figé), `Billet` (place + tarif), `EtatPlace` (libre, bloquée jusqu'à une heure, ou vendue, et par quelle commande). Tous sont des records Java immuables : on les remplace, on ne les modifie pas.

Événements et policies

Les agrégats enregistrent leurs événements ; le service les publie **après** l'enregistrement, par le port `PublicateurEvenements` (ADR-0005).

- `CommandePayee` → Notification envoie les billets ; Contrôle d'accès les ajoute à la liste des douchettes.
- `CommandeExpiree` → Abonnements rend la place gratuite au quota (B7).
- `PaiementARembourser` → Paiement rembourse par PayVia (`POST /v3/refunds`) ; Notification prévient le client.
- `VentePartenaireEnregistree` → Déclaration CNC (prix payé par le spectateur) et reversement mensuel (grille – 8 %).

9. Décisions (ADR)

ADR-0001 — Architecture retenue par contexte

Rubrique	Contenu
Statut	Accepté
Contexte	Six bounded contexts (section 4), deux développeurs et un prestataire, pile imposée (C1), mise en service avant fin 2027 (C6). La grille (section 5) sépare nettement la Vente des autres contextes. Restaient ouvertes la décision 1 (la Tarification est-elle un contexte à part ?) et la décision 2 (les Abonnements méritent-ils un DDD complet ?).
Options envisagées	• Style : hexagonal + DDD partout, ou seulement là où la grille le justifie. • D1 : (A) Tarification à part ; (B) tarification dans la Vente. • D2 : (A) DDD complet ; (B) architecture simple, formules paramétrées. • Déploiement : microservices, ou monolithe modulaire.
Décision	• **Vente** : hexagonale + DDD tactique (agrégats, ports, ACL). • **D1 → option A** : la Tarification est un contexte à part, simple (grille, suppléments, opérations saisis par le marketing), interrogé par le port `GrilleTarifaire`. Le prix est figé dans la commande (B6) : la Vente garde la garantie, la Tarification le calcul. • **D2 → option B** : Abonnements en couches (contrôleur, service, persistance), formules paramétrées, règles B7, B8, B15, B16 isolées dans un service de règles testé. Clause de revoyure : DDD si plus de 5 formules aux règles différentes, ou si l'expert devient disponible chaque semaine. • Programmation : CRUD + ACL ; Contrôle d'accès : couches + appli douchette ; Déclaration CNC : lots, Conformist ; Paiement et Notification : achetés. • Un seul déploiement Spring Boot, un package et un schéma par contexte ; communication par ports ou événements.
Conséquences positives	• L'effort de modélisation va où sont le risque et la valeur (B1, mars). • D1 : Sabine change grilles et opérations sans toucher la Vente ; ses mots (opération, supplément) restent chez elle ; le port existait déjà dans le code. • D2 : une formule se lance par paramétrage en moins d'un mois (objectif 2) ; peu de code pour deux développeurs. • Un seul déploiement : pas de réseau entre contextes, transactions simples, une seule supervision.
Conséquences négatives	• D1 : une relation de plus à maintenir, et un appel à chaque réservation. Dans le monolithe c'est un appel de méthode ; si la Tarification est un jour séparée, la Vente devra garder une copie locale de la grille pour vendre pendant une panne. • D2 : les règles d'abonnement risquent de se disperser dans les services et les requêtes ; une refonte est probable si les formules se multiplient (dette surveillée, section 11). • Monolithe : rien n'empêche techniquement un import entre contextes (→ ArchitectureTest) ; on monte en charge tout ensemble.

ADR-0002 — Agrégats et concurrence de la vente

Rubrique	Contenu
Statut	Accepté — à réévaluer après le test de charge
Contexte	B1 doit tenir au pic : 420 places vendues en moins de 5 minutes, tous canaux et partenaires confondus (l'incident de mars). Où passent les frontières des agrégats, et comment protéger B1 quand deux ventes visent la même séance au même instant (décision 3) ?
Options envisagées	• Frontières : un seul agrégat Séance contenant les commandes ; ou deux agrégats, `Seance` (plan de salle) et `Commande` (achat), référencés par identité. • Concurrence : (A) verrou optimiste sur toute la séance (version incrémentée, `ConflitDeVersion`, ré-essai) ; (B) verrou par place (une ligne par place, contrainte

Rubrique	Contenu
	d'unicité (séance, place)).
Décision	• **Deux agrégats.** `Seance` protège B1, B2, B4 et le tout ou rien ; `Commande` protège B3, B6 et son cycle de vie. Exception assumée à « un agrégat par transaction » : réserver et payer écrivent les deux dans la même transaction (sinon un blocage pourrait expirer entre la vente des places et le paiement). • **D3 → option A** : verrou optimiste par séance, avec ré-essai automatique (3 essais : recharger, faire redécider le domaine, enregistrer). • **Chiffrage.** 420 places en 5 min ≈ 1,4 place/s, soit ~0,6 commande/s (2,5 places par commande). Avec une écriture de ~50 ms, une autre commande écrit pendant la nôtre dans ~3 % des cas ; après 3 essais l'échec est négligeable. Le conflit devient gênant au-delà de ~6 commandes par seconde sur une même séance. • **Critère de bascule vers B** : plus de 1 % de `ConflitDeVersion` au test de charge (420 places en 5 min, 1 000 clients simulés) ou en production.
Conséquences positives	• B1 et le tout ou rien garantis par construction, dans le domaine, et testés sans base (SeanceTest, ConcurrenceTest). • Simple pour deux développeurs : une colonne `version` à vérifier dans PostgreSQL. • Bascule vers B sans toucher aux services : seuls l'adapter `Seances` et la frontière de l'agrégat changent.
Conséquences négatives	• Deux ventes sur des places différentes de la même séance entrent en conflit : au pic, certains clients attendent un ré-essai, quelques-uns peuvent recevoir un 409 pour une place que personne d'autre ne voulait. • La séance entière (jusqu'à 420 places) est relue et réécrite à chaque vente. • Entorse documentée à la règle de Vernon (deux agrégats par transaction). • Ce que l'option B aurait apporté : plus aucun conflit entre places différentes ; mais B1 serait descendu dans la base et le blocage de plusieurs places aurait demandé un ordre de verrouillage.

ADR-0003 — Paiement reçu deux fois ou trop tard (B13, B14)

Rubrique	Contenu
Statut	Accepté — codé (bonus)
Contexte	PayVia notifie « au moins une fois » et réessaie (1 min, 5 min, 30 min, 2 h). Son lien vit 15 min, le blocage 10 (B2) : un paiement peut arriver deux fois, ou après l'expiration de la commande.
Options envisagées	• Idempotence : table des `payment_id` déjà traités, ou statut de la commande. • Paiement tardif : toujours rembourser, ou vendre si les places sont encore libres.
Décision	• Idempotence par le statut : une commande PAYEE ou A_REMBOURSER ignore la notification (`paiementDejaTraite`) ; le webhook répond 204 pour que PayVia arrête de réessayer. • Paiement tardif : `Seance.vendre` accepte une place encore libre ; sinon la commande passe A_REMBOURSER et publie `PaiementARembourser` (B13). Même chemin si la séance a été annulée entre-temps (B10).
Conséquences positives	• Aucune table de plus ; la règle vit dans l'agrégat (PaiementEnDoubleTest, PaiementTardifTest). • Le client n'est jamais débité sans place ni remboursement.
Conséquences négatives	• Deux paiements distincts pour une même commande ne sont pas distingués → réconciliation quotidienne avec PayVia. • La signature du webhook n'est pas encore vérifiée (risque, section 11).

ADR-0004 — Ventes partenaires : une ACL par partenaire, un seul cas d'usage

Rubrique	Contenu
Statut	Accepté — codé (étape 8)
Contexte	SortiesFlash et CinéMalin envoient des ventes déjà payées, chacun dans un format qu'ils ne changeront pas (annexe B). Objectif 2 : brancher un partenaire en moins de deux semaines.
Options envisagées	• Un cas d'usage par partenaire. • Un cas d'usage commun + une ACL par partenaire. • Adopter le format de SortiesFlash dans le domaine (Conformist).
Décision	• Un seul cas d'usage `EnregistrerVentePartenaire` : mêmes règles B1–B4 que nos ventes, vente immédiate, sans blocage ni PayVia. • Une ACL par partenaire dans `adapter/in/<partenaire>/` (`AclSortiesFlash`, `CommandeSortiesFlash`, `CorrespondanceSeances`). • Le billet garde le prix de notre grille (base du reversement − 8 %) ; le montant payé chez le partenaire est conservé dans `VentePartenaireEnregistree` (CNC).
Conséquences positives	• CinéMalin = un nouveau package et ses tests, sans toucher au domaine (objectif 2). • B1 tient entre tous les canaux (VentePartenaireApiTest). • La CNC et le reversement disposent des deux montants.
Conséquences négatives	• La correspondance des séances devra être alimentée par la Programmation (codée pour la démonstration). • Pas d'idempotence sur la référence partenaire : une vente renvoyée après un délai dépassé est refusée (409) alors qu'elle est enregistrée → clé unique (partenaire, référence) à négocier.

ADR-0005 — Événements publiés après l'enregistrement, outbox en production

Rubrique	Contenu
Statut	Accepté — en mémoire au TP
Contexte	Notification, Contrôle d'accès, Abonnements et Déclaration CNC réagissent aux événements de la Vente. Un événement perdu, c'est un billet non envoyé, une douchette qui refuse un billet valide ou une déclaration fausse.
Options envisagées	• Appeler directement les autres contextes depuis les services. • Publier des événements après l'enregistrement : en mémoire, ou par une table outbox écrite dans la même transaction.
Décision	• Les agrégats enregistrent, les services publient après l'enregistrement via le port `PublicateurEvenements`. • Au TP : `PublicateurEnMemoire`. En production : un adapter outbox écrit l'événement dans la même transaction PostgreSQL que l'agrégat ; un relais le transmet, au moins une fois.
Conséquences positives	• Ajouter une réaction = une nouvelle policy, sans modifier la Vente. • Avec l'outbox, aucun événement perdu même si l'application s'arrête après la validation.
Conséquences négatives	• Livraison « au moins une fois » : chaque consommateur doit être idempotent. • Cohérence à terme : le billet arrive quelques secondes après le paiement.

10. Qualité et tests

90 tests verts (`mvn test`) : 79 en Java pur, sans Spring ni base, et 11 tests HTTP qui démarrent l'application complète. Les tests fournis n'ont pas été modifiés (seuls les `@Disabled` des bonus ont été retirés, comme demandé).

Niveau	Ce qu'on teste	Comment
Domaine	Chaque règle et cas limite des agrégats : B1, B2, B3, B4, B6, tout ou rien, cycle de vie (SeanceTest 13, CommandeTest 11)	JUnit 5 + AssertJ, Java pur ; noms de tests en langage métier ; moins d'une seconde
Application	Les cas d'usage orchestrent : enregistrer, publier après, appeler PayVia après, ne rien bloquer en cas de refus (étapes 3 à 5, bonus, EnregistrerVentePartenaireTest, ConsulterPlanTest, AnnulerSeanceTest)	Adapters en mémoire qui stockent des copies (un oubli d'enregistrement se voit) + `HorlogeReglable` : le temps est maîtrisé (B2, B4, B13)
ACL	La traduction du format SortiesFlash : types de billets, correspondance des séances, montant conservé (AclSortiesFlashTest 7)	Java pur, sans Spring
Acceptation	Le parcours complet branché par la composition root (ScenarioSoireeDuJeudiTest 2)	`ApplicationVente` + `HorlogeReglable`
API	Le contrat HTTP : 201, 409, 404, 422, 400, 204, et le format de réponse SortiesFlash (ApiVenteTest 6, VentePartenaireApiTest 5)	`@SpringBootTest` + MockMvc
Concurrence	Verrou optimiste et ré-essai (ConcurrenceTest 3)	Adapter « sous pression » qui simule une autre caisse
Architecture	Les règles de dépendance ci-dessous (etape6/ArchitectureTest 5)	Les recherches de l'étape 6, rejouées à chaque build (équivalent ArchUnit, sans dépendance)

Règles de dépendance, vérifiées à chaque build

- `domaine` ne dépend que de Java : aucune référence à `application`, `adapter`, `config` ni Spring.
- `application` ne connaît que ses ports : ni adapter, ni configuration, ni Spring.
- Les adapters primaires n'appellent que des ports, jamais un service ; aucun mot de SortiesFlash (`show_ref`, `ADULT`…) hors de son ACL.
- L'heure vient toujours de la `Clock` injectée : aucun `.now()` dans `domaine`, `application` ni `adapter`.

Étape 6 : la dépendance cachée

Ce que nous avons trouvé. La recherche `.adapter.` dans `application` renvoyait une ligne : `ReserverDesPlacesService` importait `adapter.out.tarification.GrilleTarifaireFixe` et l'utilisait comme type de champ et de paramètre. La couche application dépendait d'un adapter secondaire : la flèche partait du cœur vers l'extérieur, et brancher la vraie Tarification (décision 1) aurait obligé à modifier le service.

Comment nous l'avons corrigée. Le champ et le paramètre sont désormais typés par le port qui existait déjà, `GrilleTarifaire`. `ApplicationVente` continue d'y injecter `GrilleTarifaireFixe`. Les quatre recherches ne trouvent plus rien et tous les tests restent verts.

Pourquoi aucun test ne l'avait vue. Les tests vérifient le comportement, pas la structure : `GrilleTarifaireFixe` implémente le port, donc tout compile et se comporte de la même façon, et les tests passent eux-mêmes l'adapter concret au constructeur. Java ne contrôle pas le sens des dépendances entre packages. D'où `etape6/ArchitectureTest` : en réintroduisant l'import, il échoue (vérifié).

11. Risques et dette technique

Risque ou simplification	Impact	Parade
Conflits au pic avec le verrou par séance (ADR-0002)	Fort, un soir d'avant-première	3 ré-essais (5 avec attente aléatoire si besoin) ; test de charge avant la première avant-première ; bascule vers le verrou par place au-delà de 1 % de conflits.
Événement perdu entre l'enregistrement et la publication (publication en mémoire)	Fort : billet non envoyé, douchettes et CNC faussées	Outbox transactionnelle + relais (ADR-0005) ; consommateurs idempotents.
Webhook PayVia non authentifié	Fort : un tiers pourrait « payer » une commande	Vérifier la signature PayVia (secret partagé) dans `WebhookPayVia`, ainsi que le montant et la devise, avant la mise en production.
Notification PayVia jamais reçue, ou deux paiements pour une commande	Moyen	Réessais de PayVia + paiement tardif (B13) ; réconciliation quotidienne avec l'API PayVia.
Commande orpheline : un blocage expiré, repris par un autre client avant le passage du planificateur, laisse la commande en attente	Faible : si le paiement arrive, il est remboursé (B13)	Ajouter au port `Commandes` la recherche des commandes en attente échues, utilisée par `ExpirerBlocages`.
Équipe de 2 développeurs + prestataire, échéance fin 2027 (C6)	Fort	Cœur court et testé (90 tests), DAT et ADR à jour, contextes supporting volontairement simples, noms du code = glossaire.
Abonnements en architecture simple (D2)	Moyen : dette si les formules se multiplient	Règles B7, B8, B15, B16 isolées et testées ; clause de revoyure de l'ADR-0001.
Simplifications assumées (dette consciente)	—	• Adapters en mémoire : pas encore de PostgreSQL ; grille fixe sans suppléments ni opérations ; paiement simulé. • Séance S42 et correspondance SortiesFlash codées dans `VenteConfiguration` (pas de Programmation). • Non codés dans la Vente : PMR (B5), droits d'abonné (B7, B15, B16), annulation par le client (B9). • Pas d'authentification sur l'API ; pas d'idempotence sur la référence partenaire (ADR-0004).